ASSIGNMENT – 5

1. Student Marks Input Validation

```
import numpy as np


names = ['A','B','C,'D','E','F,'G','H','I']

subjects = ['Math','Science','English']


marks_2d = np.array([ [85,80,88],[90,85,92],[78,72,75],[92,88,90],[67,70,65],
[88,83,86],[74,78,80],[81,76,79],[95,91,93] ])


total = [marks_2d[i].sum() for i in range(len(names))]

[print(f'{n}: {t}') for n, t in zip(names, total)]


avg = [round(marks_2d[:, j].mean(), 2) for j in range(3)]

[print(f'{s}: {a}') for s, a in zip(subjects, avg)]


print(f'Top student: {names[np.argmax(total)]} ({max(total)} marks)')
```

```
A: 253
B: 267
C: 225
D: 270
E: 202
F: 257
G: 232
H: 236
I: 279
Math: 83.33
Science: 80.33
English: 83.11
Top student: I (279 marks)
```

2. Data Filtering

```
import pandas as pd


df = pd.DataFrame({
    'Name': ['A', 'B', 'C', 'D', 'E', 'F', 'G'],
    'Marks': [85, 62, 91, 45, 78, 55, 88],
    'City': ['City1', 'City2', 'City2', 'City3', 'City4', 'City5', 'City6']
})


above_70 = df[df['Marks'] > 70]


sorted_df = df.sort_values('Marks', ascending=False)


def assign_grade(m):
    if m >= 90: return 'A+'
    elif m >= 80: return 'A'
    elif m >= 70: return 'B'
    elif m >= 60: return 'C'
    else: return 'D'


df['Grade'] = df['Marks'].apply(assign_grade)
print(df)
```

```
Values > 50 : [72, 62, 76, 72, 91, 57, 86, 69, 73, 57, 99]
Even numbers : [72, 62, -16, 76, -10, 72, 44, 86, 22, -10]
Negatives > 0 : [72, 62, 0, 76, 41, 0, 72, 91, 44, 57, 86, 69, 73, 22, 0, 57, 7, 99, 0, 27]
```

3. Student DataFrame

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['A', 'B', 'C', 'D', 'E', 'F', 'G'],
    'Marks': [85, 62, 91, 45, 78, 55, 88],
    'City': ['City1', 'City2', 'City2', 'City3', 'City4', 'City5', 'City6']
})

above_70 = df[df['Marks'] > 70]

sorted_df = df.sort_values('Marks', ascending=False)

def assign_grade(m):
    if m >= 90: return 'A+'
    elif m >= 80: return 'A'
    elif m >= 70: return 'B'
    elif m >= 60: return 'C'
    else: return 'D'

df['Grade'] = df['Marks'].apply(assign_grade)
print(df)
```

	Name	Marks	City	Grade
0	A	85	City1	A
1	B	62	City2	C
2	C	91	City2	A+
3	D	45	City3	D
4	E	78	City4	B
5	F	55	City5	D
6	G	88	City6	A

4. Employee Analysis

```
import pandas as pd


df_emp  = pd.DataFrame({
    'Name': ['A', 'B', 'C', 'D', 'E', 'F', 'G'],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'HR', 'Finance', 'IT'],
    'Salary': [75000, 50000, 90000, 65000, 48000, 70000, 80000]
})


avg_sal = df_emp.groupby('Department')['Salary'].mean()


top = df_emp.loc[df_emp['Salary'].idxmax()]
print(f'Highest paid: {top["Name"]} - {top["Salary"]}')


count = df_emp.groupby('Department')['Name'].count()
print(count)
```

```
Highest paid: C - 90000
Department
Finance    2
HR         2
IT         3
Name: Name, dtype: int64
```

5. Missing Data Handling

```
import pandas as pd

df = pd.read_csv("data.csv")

print("Original Data:")
print(df)

print("\nMissing Values per Column:")
print(df.isnull().sum())

df_filled = df.fillna(df.mean(numeric_only=True))
print("\nData after filling missing values:")
print(df_filled)

df_dropped = df.dropna()
print("\nData after dropping missing rows:")
print(df_dropped)
```

```
Original Data:
  Name   Age  Salary  Marks
0    A    25   50000     85
1    B    30   60000     90
2    C    35   70000    NaN
3    D    40   80000     95
4    E    45     NaN     88

Missing Values per Column:
Name       0
 Age       0
 Salary    0
 Marks     0
dtype: int64

Data after filling missing values:
  Name   Age  Salary  Marks
0    A    25   50000     85
1    B    30   60000     90
2    C    35   70000    NaN
3    D    40   80000     95
4    E    45     NaN     88

Data after dropping missing rows:
  Name   Age  Salary  Marks
...
1    B    30   60000     90
2    C    35   70000    NaN
3    D    40   80000     95
4    E    45     NaN     88
```

6. Load CSV & Basic Operations

```
import pandas as pd


df = pd.read_csv("data2.csv")

df.columns = df.columns.str.strip()


print("Top 5 Rows:")

print(df.head())


print("\nUnique Values in Category:")

print(df['Category'].unique())


print("\nGrouped Data (Category-wise Count):")

print(df.groupby('Category').size())
```

```
Top 5 Rows:
  Name      Category          City  Sales
0    A   Electronics      New York   1000
1    B      Clothing   Los Angeles   1500
2    C   Electronics       Chicago   2000
3    D      Clothing       Houston   1200
4    E   Electronics       Phoenix   1800

Unique Values in Category:
[' Electronics' ' Clothing' ' Grocery']

Grouped Data (Category-wise Count):
Category
Clothing       3
Electronics    4
Grocery        3
dtype: int64
```

7. Data Aggregation

```
import pandas as pd


data = {
    'Name': ['A', 'B', 'C', 'D', 'E', 'F', 'G'],
    'City': ['Mumbai', 'Pune', 'Nagpur', 'Mumbai', 'Pune', 'Nagpur', 'Mumbai'],
    'Salary': [75000, 55000, 45000, 85000, 60000, 48000, 90000],
    'Age': [22, 24, 23, 25, 22, 26, 24]
}
df = pd.DataFrame(data)


result = df.groupby('City').agg({
    'Salary': 'mean',
    'Name': 'count'
})
result.rename(columns={'Salary':  'Average Income', 'Name': 'Total Population'}, inplace=True)
print(result)
```

```
         Average Income  Total Population
City
Mumbai    83333.333333                 3
Nagpur    46500.000000                 2
Pune      57500.000000                 2
```

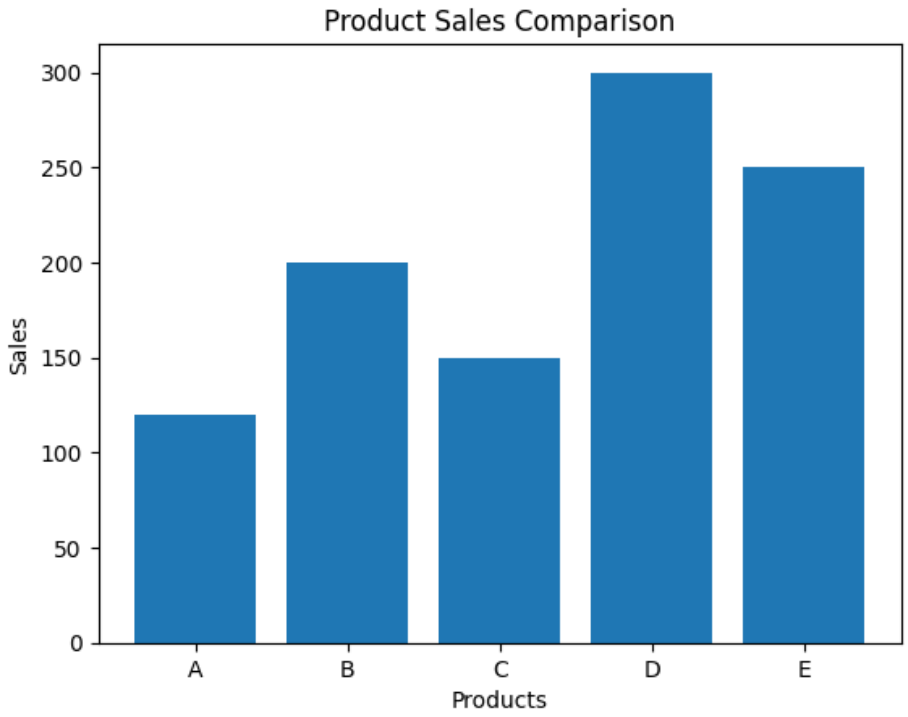
9. Merging DataFrames

```
import pandas as pd


students = {
   'ID': [1, 2, 3, 4],
   'Name': ['A', 'B', 'C', 'D']
}
df_students = pd.DataFrame(students)


marks = {
   'ID': [1, 2, 3, 4],
   'Marks': [88, 76, 91, 65]
}
df_marks = pd.DataFrame(marks)


merged = pd.merge(df_students, df_marks, on='ID')
print(merged)
```

```
   ID Name  Marks
0   1    A     88
1   2    B     76
2   3    C     91
3   4    D     65
```

11. Product Comparison

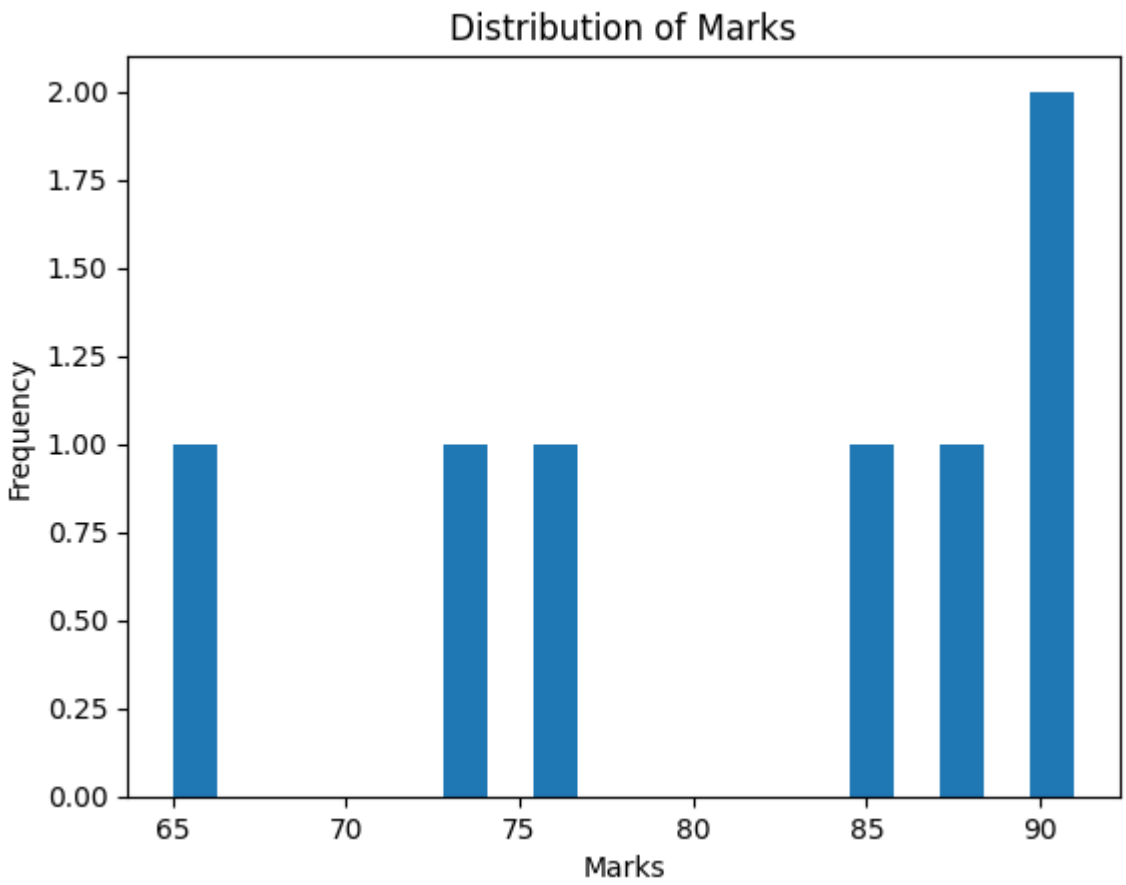
```
import matplotlib.pyplot as plt


products = ['A', 'B', 'C', 'D', 'E']

sales = [120, 200, 150, 300, 250]


plt.bar(products, sales)

plt.title("Product Sales Comparison")

plt.xlabel("Products")

plt.ylabel("Sales")

plt.show()


max_sales = max(sales)

index = sales.index(max_sales)

print("Highest Selling Product:", products[index])
```



Highest Selling Product: D

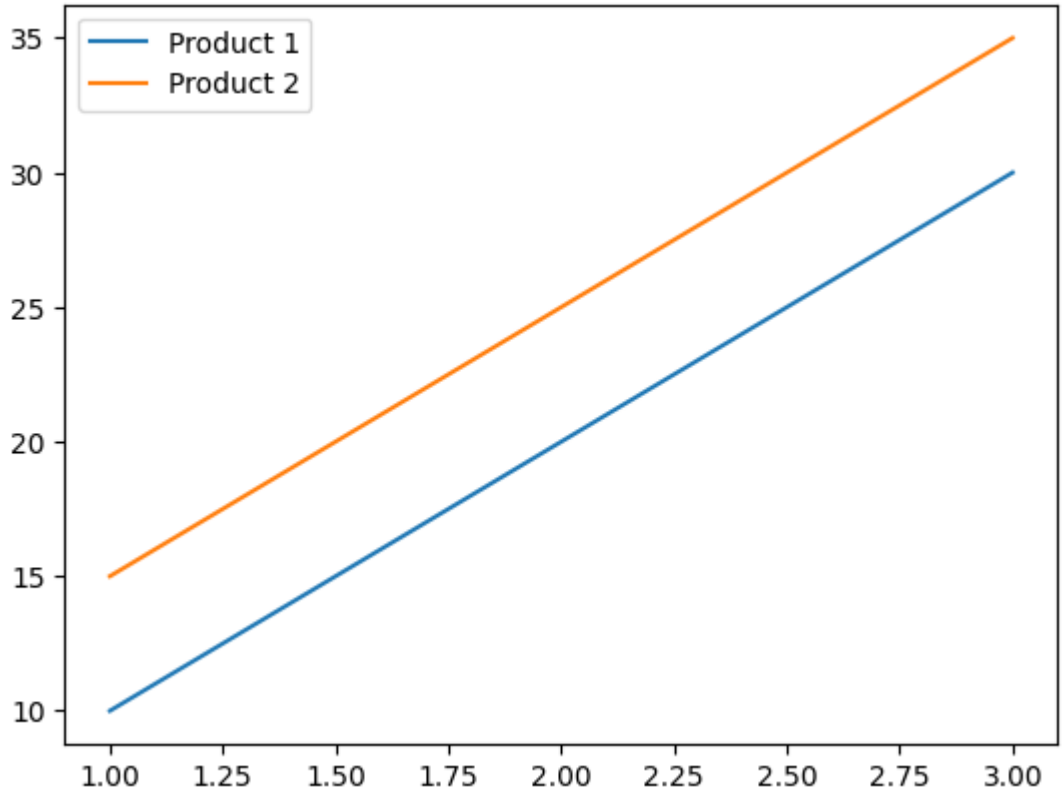
12. Distribution Plot

```
import pandas as pd

import matplotlib.pyplot as plt


data = {

   'Name': ['A', 'B', 'C', 'D', 'E', 'F', 'G'],

   'City': ['Mumbai', 'Pune', 'Nagpur', 'Mumbai', 'Pune', 'Nagpur', 'Mumbai'],

   'Salary': [75000, 55000, 45000, 85000, 60000, 48000, 90000],

   'Age': [22, 24, 23, 25, 22, 26, 24],

   'Marks': [88, 76, 91, 65, 73, 85, 90]

}

df = pd.DataFrame(data)


plt.hist(df['Marks'], bins=20)

plt.title("Distribution of Marks")

plt.xlabel("Marks")

plt.ylabel("Frequency")

plt.show()
```



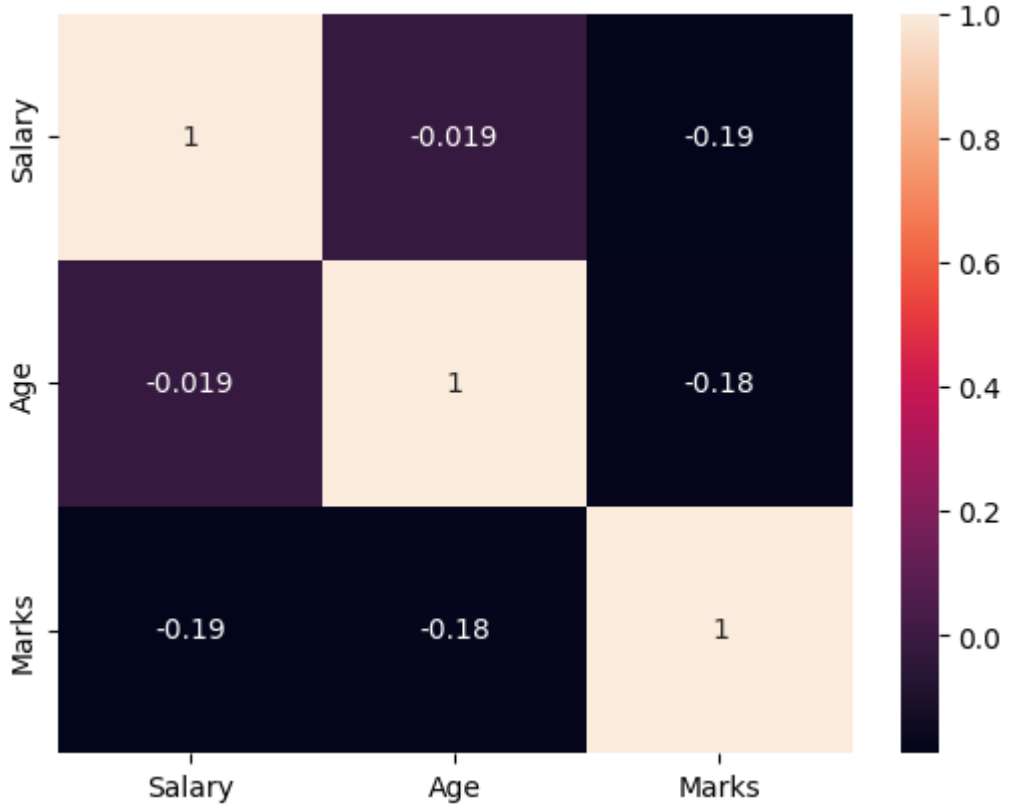
13. Multi-line Graph

```
import matplotlib.pyplot as plt


x = [1, 2, 3]

product1 = [10, 20, 30]

product2 = [15, 25, 35]


plt.plot(x, product1, label="Product 1")

plt.plot(x, product2, label="Product 2")

plt.legend()

plt.show()
```



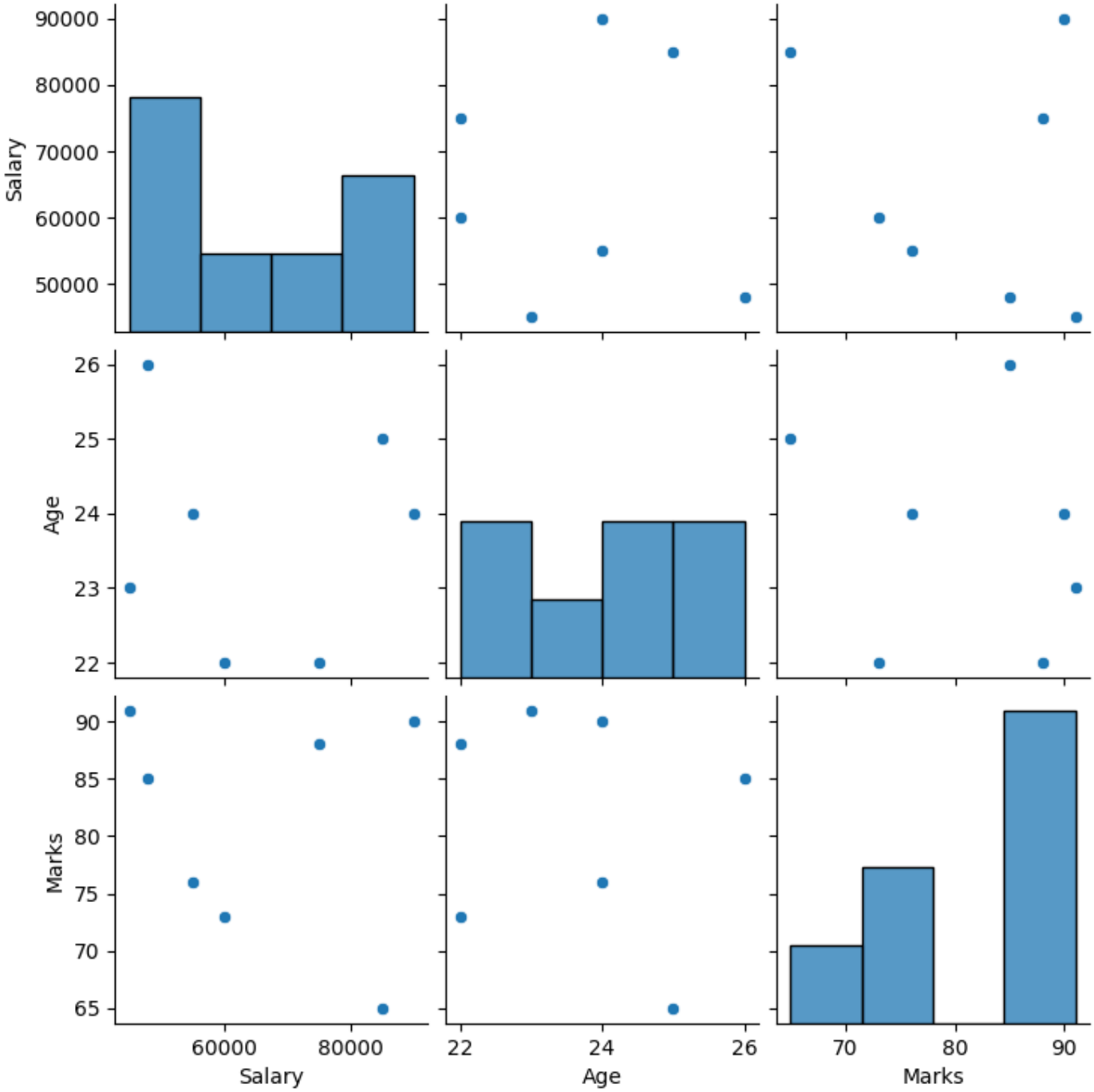
14. Heatmap

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

data = {
   'Name': ['A', 'B', 'C', 'D', 'E', 'F', 'G'],
   'City': ['Mumbai', 'Pune', 'Nagpur', 'Mumbai', 'Pune', 'Nagpur', 'Mumbai'],
   'Salary': [75000, 55000, 45000, 85000, 60000, 48000, 90000],
   'Age': [22, 24, 23, 25, 22, 26, 24],
   'Marks': [88, 76, 91, 65, 73, 85, 90]
}
df = pd.DataFrame(data)

corr = df[['Salary', 'Age', 'Marks']].corr()
sns.heatmap(corr,  annot=True)
plt.show()
```



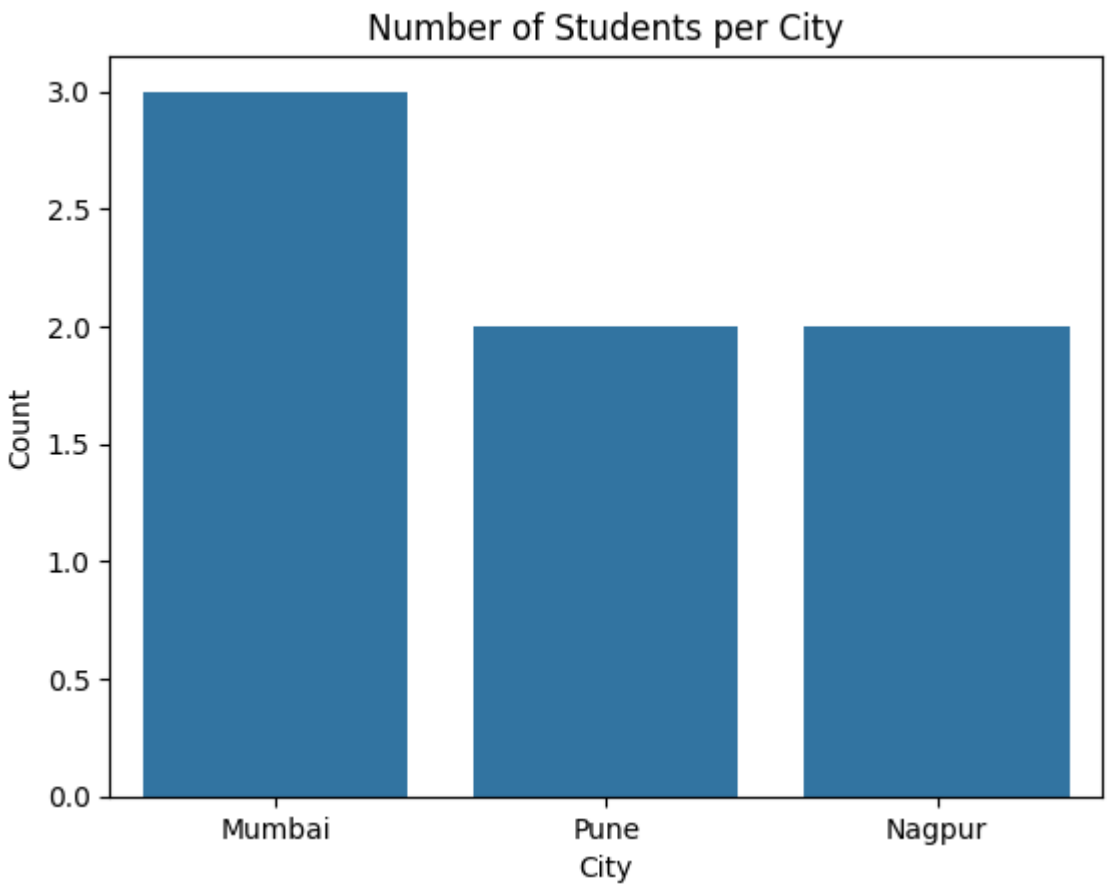
16. Pair Plot

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt


data = {

   'Name': ['A', 'B', 'C', 'D', 'E', 'F', 'G'],

   'City': ['Mumbai', 'Pune', 'Nagpur', 'Mumbai', 'Pune', 'Nagpur', 'Mumbai'],

   'Salary': [75000, 55000, 45000, 85000, 60000, 48000, 90000],

   'Age': [22, 24, 23, 25, 22, 26, 24],

   'Marks': [88, 76, 91, 65, 73, 85, 90]

}

df = pd.DataFrame(data)


sns.pairplot(df)

plt.show()
```


Salary
Age
Marks
90000
80000
70000
60000
50000
26
25
24
23
22
90
85
80
75
70
65
60000
80000
22
24
26
70
80
90

17. Count Plot

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
   'Name': ['A', 'B', 'C', 'D', 'E', 'F', 'G'],
   'City': ['Mumbai', 'Pune', 'Nagpur', 'Mumbai', 'Pune', 'Nagpur', 'Mumbai'],
   'Salary': [75000, 55000, 45000, 85000, 60000, 48000, 90000],
   'Age': [22, 24, 23, 25, 22, 26, 24],
   'Marks': [88, 76, 91, 65, 73, 85, 90]
}
df = pd.DataFrame(data)

sns.countplot(x='City', data=df)
plt.title("Number of Students per City")
plt.xlabel("City")
plt.ylabel("Count")
plt.show()
```



1. Student Management System (CRUD)

```
import mysql.connector

conn = mysql.connector.connect(host="localhost", user="root", password="brucewayne")
cursor = conn.cursor()

cursor.execute("CREATE DATABASE IF NOT EXISTS student_db")
cursor.execute("USE student_db")

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  age INT,
  course VARCHAR(100)
)
""")

students_data = [
  ('A', 22, 'Data Science'),
  ('B', 24, 'Python'),
  ('C', 23, 'AI'),
  ('D', 25, 'ML'),
  ('E', 22, 'SQL')
]

cursor.executemany("INSERT INTO students (name, age, course) VALUES (%s, %s, %s)", students_data)
conn.commit()

print("All Students:")
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
  print(row)
```

```
cursor.execute("UPDATE students SET course='Deep Learning' WHERE name='B'")
conn.commit()

cursor.execute("DELETE FROM students WHERE id=3")
conn.commit()

name = 'A'
cursor.execute("SELECT * FROM students WHERE name=%s", (name,))
print("\nSearch Result:")
print(cursor.fetchall())

cursor.close()
conn.close()
```

```
(.venv) PS D:\CDAC\2. Python> & "d:\C
thon.exe" "d:/CDAC/2. Python/Assignme
All Students:
(1, 'A', 22, 'Data Science')
(2, 'B', 24, 'Python')
(3, 'C', 23, 'AI')
(4, 'D', 25, 'ML')
(5, 'E', 22, 'SQL')

Search Result:
[(1, 'A', 22, 'Data Science')]
(.venv) PS D:\CDAC\2. Python>
```

2. Employee Database

```
import mysql.connector

conn = mysql.connector.connect(host="localhost", user="root", password="brucewayne")
cursor = conn.cursor()

cursor.execute("CREATE DATABASE IF NOT EXISTS company_db")
cursor.execute("USE company_db")

cursor.execute("""
CREATE TABLE IF NOT EXISTS employee (
    emp_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name VARCHAR(100),
    department VARCHAR(100),
    salary FLOAT
)
""")
conn.commit()

while True:
    print("\n--- Employee Menu ---")
    print("1. Add Employee")
    print("2. View All Employees")
    print("3. Update Salary")
    print("4. Delete Employee")
    print("5. Highest Paid Employee")
    print("6. Count Employees per Department")
    print("7. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        name = input("Enter name: ")
        dept = input("Enter department: ")
        salary = float(input("Enter salary: "))
        cursor.execute("INSERT INTO employee (name, department, salary) VALUES (%s, %s, %s)", (name, dept, salary))
        conn.commit()
        print("Employee added successfully!")

    elif choice == 2:
        cursor.execute("SELECT * FROM employee")
        for row in cursor.fetchall():
            print(row)

    elif choice == 3:
```

```
        emp_id = int(input("Enter employee ID: "))
        new_salary = float(input("Enter new salary: "))
        cursor.execute("UPDATE employee SET salary=%s WHERE emp_id=%s", (new_salary, emp_id))
        conn.commit()
        print("Salary updated!")

    elif choice == 4:
        emp_id = int(input("Enter employee ID to delete: "))
        cursor.execute("DELETE FROM employee WHERE emp_id=%s", (emp_id,))
        conn.commit()
        print("Employee deleted!")

    elif choice == 5:
        cursor.execute("SELECT * FROM employee ORDER BY salary DESC LIMIT 1")
        print("Highest Paid Employee:")
        print(cursor.fetchone())

    elif choice == 6:
        cursor.execute("SELECT department, COUNT(*) FROM employee GROUP BY department")
        print("Employees per Department:")
        for row in cursor.fetchall():
            print(row)

    elif choice == 7:
        print("Exiting program...")
        break

    else:
        print("Invalid choice!")

cursor.close()
conn.close()
```

```
(.venv) PS D:\CDAC\2. Python> & "d:\CDAC\2. Pyth
thon.exe" "d:/CDAC/2. Python/Assignment 5/Db2ass

--- Employee Menu ---
1. Add Employee
2. View All Employees
3. Update Salary
4. Delete Employee
5. Highest Paid Employee
6. Count Employees per Department
7. Exit
Enter choice: 1
Enter name: A
Enter department: Education
Enter salary: 10000
Employee added successfully!

--- Employee Menu ---
1. Add Employee
2. View All Employees
3. Update Salary
4. Delete Employee
5. Highest Paid Employee
6. Count Employees per Department
7. Exit
Enter choice: 2
(1, 'A', 'Education', 10000.0)
```

```
--- Employee Menu ---
1. Add Employee
2. View All Employees
3. Update Salary
4. Delete Employee
5. Highest Paid Employee
6. Count Employees per Department
7. Exit
Enter choice: 3
Enter employee ID: 1
Enter new salary: 50000
Salary updated!

--- Employee Menu ---
1. Add Employee
2. View All Employees
3. Update Salary
4. Delete Employee
5. Highest Paid Employee
6. Count Employees per Department
7. Exit
Enter choice: 6
Employees per Department:
('Education', 1)

--- Employee Menu ---
1. Add Employee
2. View All Employees
3. Update Salary
4. Delete Employee
5. Highest Paid Employee
6. Count Employees per Department
7. Exit
Enter choice:
```

```
--- Employee Menu ---
1. Add Employee
2. View All Employees
3. Update Salary
4. Delete Employee
5. Highest Paid Employee
6. Count Employees per Department
7. Exit
Enter choice: 4
Enter employee ID to delete: 1
Employee deleted!

--- Employee Menu ---
1. Add Employee
2. View All Employees
3. Update Salary
4. Delete Employee
5. Highest Paid Employee
6. Count Employees per Department
7. Exit
Enter choice:
```